Ex 4-2

1/30/24 08:01

Exercise 4.2

Write a program which uses BLAS to compute the product, C_1, of two matrices A and B of size $N \times N$, where $N = 2^n$, for some integer $n > 0$. Write an additional function which performs the calculation manually, storing in matrix C_2.

Calculate the difference, using BLAS, between C_1 and C_2 to verify your solutions match:

$$\epsilon = \sum_{i=1}^{N} \sum_{j=1}^{N} |C_1[i][j] - C_2[i][j]|$$

```
#include <iostream>
#include <cstdlib>
using namespace std;
#include "cblas.h"
// Naively multiply the two matrices A and B into the matrix C
void multiply(int n, double* A, double* B, double* C) {
    for (int i = 0; i < n; i++) {           // Columns of C
        for (int j = 0; j < n; j++) {       // Rows of C
            C[i*n+j] = 0.0;
            for (int k = 0; k < n; k++) {   // Along row of A / Column of B
                C[i*n+j] += A[i*n+k] * B[k*n+j];
            }
        }
    }
}
// Call general matrix-matrix multiplication BLAS routine
void multiplyBLAS(int n, double* A, double* B, double* C) {
    cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                n, n, n, 1.0, A, n, B, n, 0.0, C, n);
}
// Print a matrix
void printMatrix(int n, double* A) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << A[i*n+j] << "  ";
        }
        cout << endl;
    }
}
int main() {
    const int n = 2;
    const int range = 5.0;
    // Create three matrices of size n
    double* A  = new double[n*n];
    double* B  = new double[n*n];
    double* C1 = new double[n*n];
    double* C2 = new double[n*n];
    // Seed the random number generator and fill A and B matrices
    srand(time(0));
    for (int i = 0; i < n*n; i++) {
        A[i]  = (double)rand()/RAND_MAX*range;
        B[i]  = (double)rand()/RAND_MAX*range;
        C1[i] = 0.0;
        C2[i] = 0.0;
    }
    // Print the matrices
    cout << "Matrix A: " << endl;
    printMatrix(n,A);
    cout << "Matrix B: " << endl;
    printMatrix(n,B);
    // Multiply A and B into D using BLAS
    multiplyBLAS(n, A, B, C1);
    // Multiply A and B into D using manual routine
    multiply    (n, A, B, C2);
    // Print C and D
    cout << "Matrix C1 = A x B (BLAS): " << endl;
    printMatrix(n, C1);
    cout << "Matrix C2 = A x B (manual): " << endl;
    printMatrix(n, C2);
    // Calculate difference using y <- ax + y
    // Then compute the norm
    // Note that we now interpret the matrices C1 and C2 eac as a single column
    // vector, since they are stored contiguously column-by-column in memory.
    cblas_daxpy(n*n, -1.0, C2, 1, C1, 1);
    double eps = cblas_dnrm2(n*n, C1, 1);
    cout << "Epsilon: " << eps << endl;
    // Free dynamic memory
    delete[] A;
    delete[] B;
    delete[] C1;
    delete[] C2;
    return 0;
}
```